

Tests avec React

Différentes options

- React Testing Library (Utilise Jest)
- Enzyme
- Cypress
- Puppeteer
- Jasmine

Code Github

- Référence: <https://testing-library.com/docs/react-testing-library/intro/>
- Référence: React Testing Library Tutorial (youtube) de Net Ninja
<https://www.youtube.com/playlist?list=PLxj0kbFyJfCl5oGV3li8wjyg76lhqEQt6>
- `git clone https://github.com/cegep-Andre-Laurendeau1/react-testing-library.git`
- `cd React-Testing-Library`
- `npm install + npm run dev`
- MIS À JOUR AVEC VITE ET REACT 19

Smoke Test

- npm start (Si erreur de ssl, inclure le script suivant) Node 17 vs Node 16

```
"scripts": {  
  "start": "react-scripts --openssl-legacy-provider start",  
  "build": "react-scripts build",  
  "test": "react-scripts test",  
  "eject": "react-scripts eject"  
}
```

- Explore application Todo
- Explore package.json
- npm test
- Structure des tests (Component.test.js)
- 'test' vs 'it'

Test block

- Voir App.test.js. Test qui vient avec la création d'un nouveau projet React
- Répertoire __test__

Test block

- Render un component que l'on veut tester

- ```
render(<App />);
```

- Cherche un élément avec lequel on veut interagir

- ```
const linkElement = screen.getByText(/learn react/i);
```

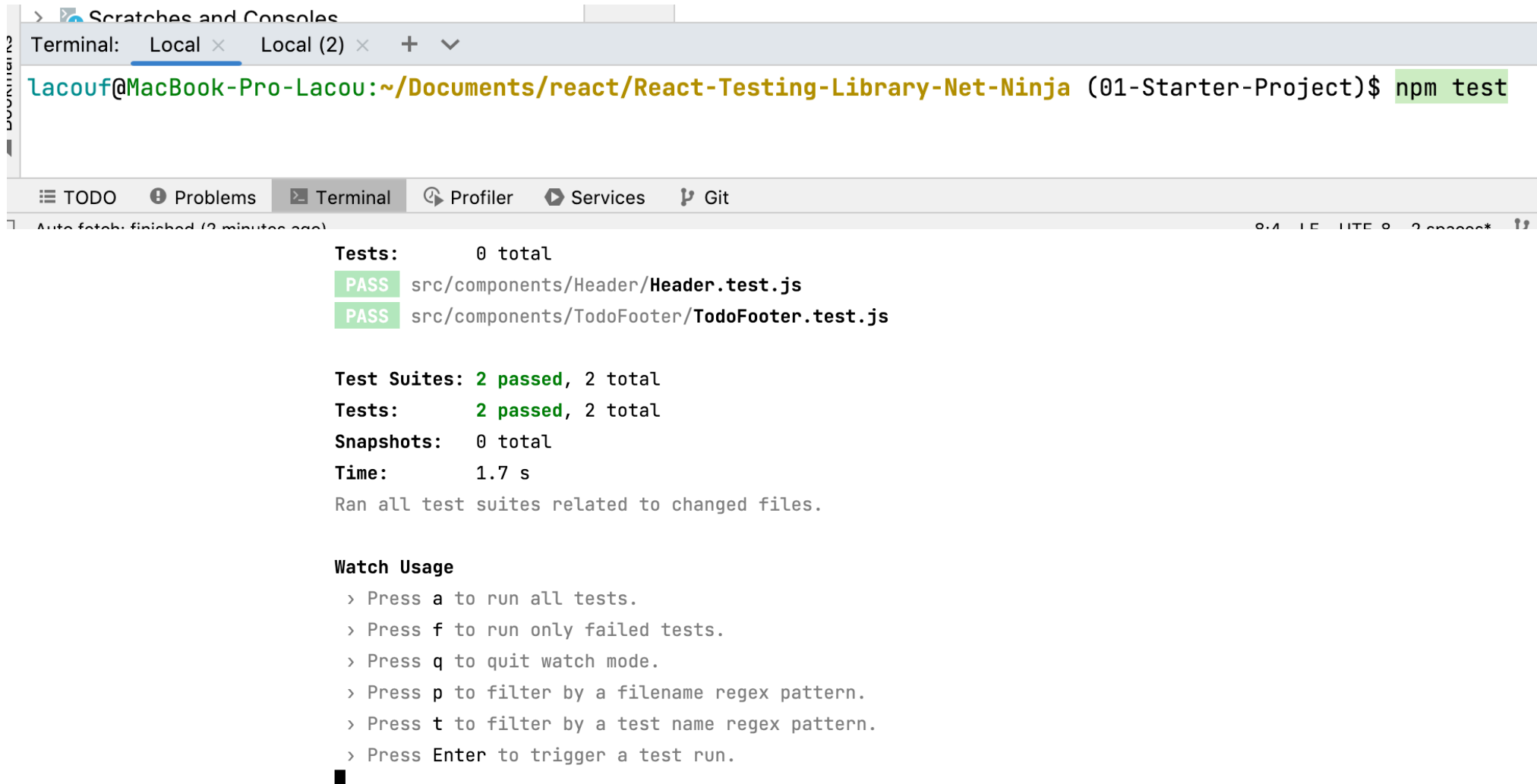
- Interagir avec l'élément

- Assertions

- ```
expect(linkElement).toBeInTheDocument();
```

# React-testing-library

- Pour rouler les tests, on utilise la commande 'npm test' dans un terminal.



```
> Scratches and Consoles
Terminal: Local x Local (2) x + v
Lacouf@MacBook-Pro-Lacou:~/Documents/react/React-Testing-Library-Net-Ninja (01-Starter-Project)$ npm test

Tests: 0 total
PASS src/components/Header/Header.test.js
PASS src/components/ToDoFooter/ToDoFooter.test.js

Test Suites: 2 passed, 2 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 1.7 s
Ran all test suites related to changed files.

Watch Usage
 > Press a to run all tests.
 > Press f to run only failed tests.
 > Press q to quit watch mode.
 > Press p to filter by a filename regex pattern.
 > Press t to filter by a test name regex pattern.
 > Press Enter to trigger a test run.
```

# Recherche d'éléments

- avec l'objet 'screen'
- get / find / query / getAll / findAll / queryAll
- <https://testing-library.com/docs/queries/about>

|          | getBy  | findBy | queryBy | getAllBy | findAllBy | queryAllBy |
|----------|--------|--------|---------|----------|-----------|------------|
| No Match | error  | error  | null    | error    | error     | array      |
| 1 Match  | return | return | return  | array    | array     | array      |
| 1+ Match | error  | error  | error   | array    | array     | array      |
| Await    | no     | yes    | no      | no       | yes       | no         |

# Différents query

Positionner après le 'get' et faire 'ctrl-espace' dans IntelliJ

```
screen.get("heading", {name: "My Header"})
.toBe
```

- getByRole (@testing-library/dom)
- getText <W>(text: (((content: string, element
- getAllByPlaceholderText <W>(text: (((content: s
- getAllByDisplayValue <W>(text: (((content: stri
- getAllByLabelText <W>(text: (((content: string,
- getAllByAltText <W>(text: (((content: string, e
- getAllByRole (@testing-library/dom)
- getAllByTestId <W>(text: (((content: string, el
- getAllByText <W>(text: (((content: string, elem
- getAllByTitle <W>(text: (((content: string, ele
- getByAltText <W>(text: (((content: string, elem
- getByDisplayValue <W>(text: (((content: string,

Press ↵ to insert, → to replace

# Examples

```
import React from 'react'
import './Header.css'

export default function Header({
 title
}) {
 return <h1 className="header">{title}</h1>
}
```

```
import {render, screen} from '@testing-library/react';
import Header from './Header';

it(name: 'should render same text passed into title prop', fn: async () => {
 render(<Header title="My Header"/>);
 const headingElement = screen.getByRole(text: "heading", options: {name: "My Header"})
 expect(headingElement).toBeInTheDocument();
});
```

# Exemple brisé

```
import {render, screen} from "@testing-library/react";
import TodoFooter from "../TodoFooter";
import {BrowserRouter} from "react-router-dom";

it(name: 'should render the correct amount of incomplete tasks', fn: async () => {
 render(<TodoFooter numberOfIncompleteTasks={5}/>);
 const paragraphElement = screen.getByText(text: /5 tasks left/i);
 expect(paragraphElement).toBeInTheDocument();
});
```

```
function TodoFooter({Harb, 2021-06-19, 19:03 • Complete starter project
 numberOfIncompleteTasks
}) {
 return (
 <div className="todo-footer">
 <p>{numberOfIncompleteTasks} {numberOfIncompleteTasks === 1 ? "task" : "tasks"} left</p>
 <Link to="/followers">Followers</Link>
 </div>
)
}
```

# Exemple brisé

- should render the correct amount of incomplete tasks

Invariant failed: You should not use <Link> outside a <Router>

```
4 |
5 | it('should render the correct amount of incomplete tasks', async () => {
> 6 | render(<TodoFooter numberOfIncompleteTasks={5}/>);
 | ^
7 | const paragraphElement = screen.getByText(/5 tasks left/i);
8 | expect(paragraphElement).toBeInTheDocument();
9 | }
```

# Exemple brisé (fix)

```
const WrappingComponent = ({numberOfIncompleteTasks}) => {
 return (
 <BrowserRouter>
 <TodoFooter numberOfIncompleteTasks={numberOfIncompleteTasks}/>
 </BrowserRouter>
)
}

it(name: 'should render the correct amount of incomplete tasks', fn: async () => {
 render(<WrappingComponent numberOfIncompleteTasks={5}/>);
 const paragraphElement = screen.getByText(text: /5 tasks left/i);
 expect(paragraphElement).toBeInTheDocument();
}
);
```

# Assertions

```
it(name: 'should render "task" when the number', fn: async () => {
 render(<WrappingComponent numberOfIncompleteTasks={1}/>);
 const paragraphElement = screen.getByText(text: /1 task left/i);
 expect(paragraphElement).to;
}
);
```

 **toBeInTheDocument()** (jest.Matchers, testing-library\_\_jest-dom)

 **toMatchInlineSnapshot**

 **toBeCalled()** (jest.Matchers, jest)

 **toContainElement(element: HTMLElement | SVGElement | null)** (jest.Matchers, te...

 **toBeGreaterThanOrEqual(expected: number | bigint)** (jest.Matchers, jest)

 **toBe(expected: E)** (jest.Matchers, jest)

 **toBeCalledTimes(expected: number)** (jest.Matchers, jest)

 **toBeCalledWith(...args: E)** (jest.Matchers, jest)

 **toBeChecked()** (jest.Matchers, testing-library\_\_jest-dom)

 **toBeCloseTo(expected: number, numDigits?: number)** (jest.Matchers, jest)

 **toBeDefined()** (jest.Matchers, jest)

 **toBeDisabled()** (jest.Matchers, testing-library\_\_jest-dom)

Press ↵ to insert, → to replace [Next Tip](#)

[react-dom.development.  
modules/react-dom/cjs/](#)

# Assertions

```
30 // ...
31 const paragraphElement = screen.getByText(text: /1 task left/i);
32 expect(paragraphElement).toBe;
```

jest.Matchers<R,T>.toBeTruthy(): R

Use when you don't care what a value is, you just want to ensure a value is true in a boolean context. In JavaScript, there are six falsy values: false, 0, '', null, undefined, and NaN. Everything else is truthy.

jest



|  |                                                                         |      |
|--|-------------------------------------------------------------------------|------|
|  | toBe(expected: E) (jest.Matchers, jest)                                 | void |
|  | toBeInTheDocument() (jest.Matchers, testing-library__jest-dom)          | void |
|  | toBeTruthy() (jest.Matchers, jest)                                      | void |
|  | toBeCalledWith(...args: E) (jest.Matchers, jest)                        | void |
|  | toBeCalledTimes(expected: number) (jest.Matchers, jest)                 | void |
|  | toBeCalled() (jest.Matchers, jest)                                      | void |
|  | toBeChecked() (jest.Matchers, testing-library__jest-dom)                | void |
|  | toBeCloseTo(expected: number, numDigits?: number) (jest.Matchers, jest) | void |
|  | toBeDefined() (jest.Matchers, jest)                                     | void |
|  | toBeDisabled() (jest.Matchers, testing-library__jest-dom)               | void |
|  | toBeEmpty() (jest.Matchers, testing-library__jest-dom)                  | void |
|  | toBeEmptyDOMElement() (jest.Matchers, testing-library__jest-dom)        | void |

Press to insert, to replace Next Tip

```
dispatchEvent (node_modules/jsdom/lib/jsdom/l
atchEvent (node_modules/jsdom/lib/jsdom/l
```

# Assertions

- On peut inverser les assertions en préfixant 'not'

```
it(name: 'should render "task" when the number', fn: async () => {
 render(<WrappingComponent numberOfIncompleteTasks={1}/>);
 const paragraphElement = screen.getByText(text: /1 task left/i);
 expect(paragraphElement).not.toBeVisible();
}
);
```

# Describe

- Permet de grouper les tests (test suites)

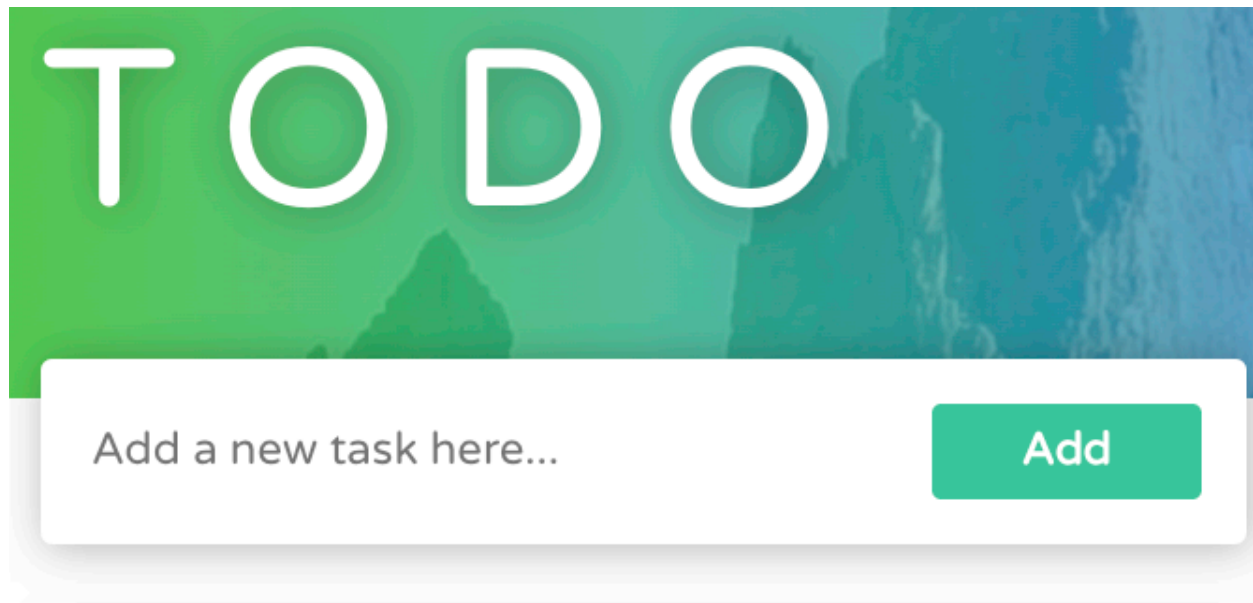
```
import {render, screen} from "@testing-library/react";
import Header from "../Header";

describe('tests de header', () => {

 it('should render same text passed into title prop', async () => {
 render(<Header title="My Header"/>);
 const headingElement = screen.getByRole({text: "heading", options: {name: "My Header"}});
 expect(headingElement).toBeInTheDocument();
 });
});
```

# Test d'événements

- Comment on test les événements d'une composante



- On aimerait pouvoir tester que quand on tape au clavier que les touches s'inscrivent dans le input field.
- Et que quand on click sur Add, que le Input field se vide

# FireEvent

- La méthode 'fireEvent' fourni par 'react-testing-library' permet de faire celà.

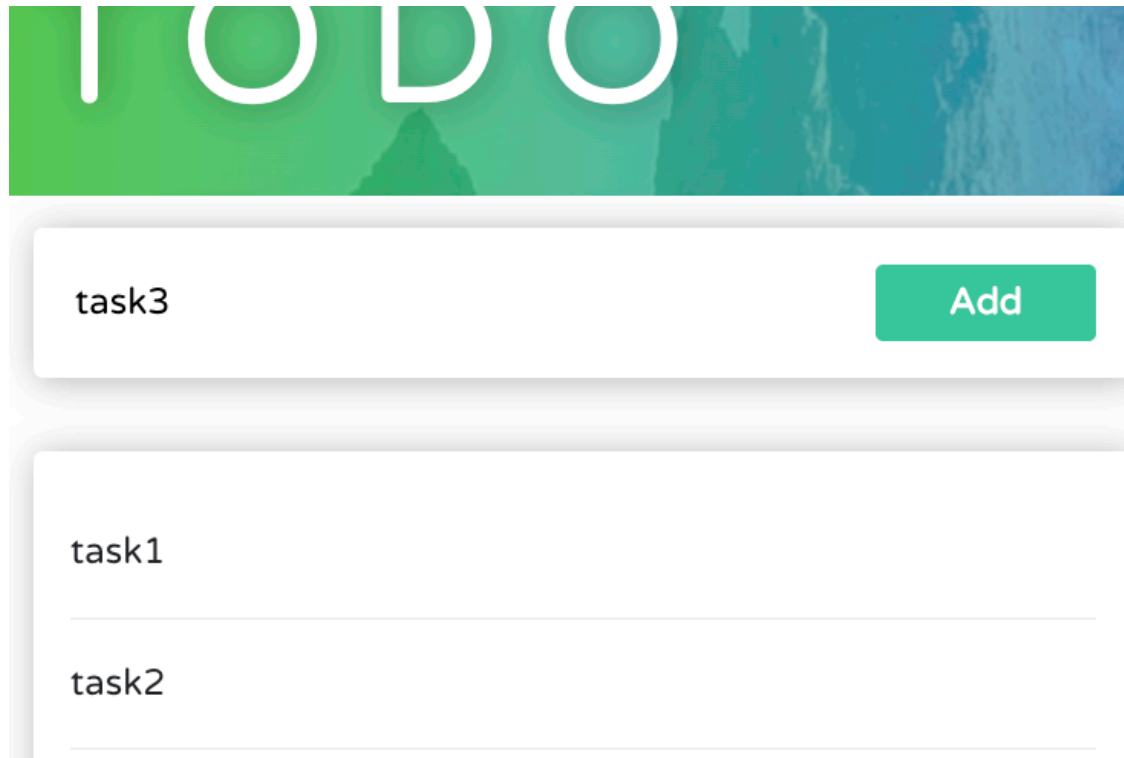
```
it(name: 'should be able to type in input', fn: async () => {
 render(<AddInput
 todos={[]}
 setTodos={mockedSetTodo}
 />);
 const inputElement = screen.getByPlaceholderText(text: /add a new task here.../i)
 fireEvent.change(inputElement, options: {target: {value: "Go Grocery Shopping"}})
 expect(inputElement.value).toBe(expected: "Go Grocery Shopping");
}
);
```

# FireEvent

```
it(name: 'should have empty input when add button is clicked', fn: async () => {
 render(<AddInput
 todos={[]}
 setTodos={mockedSetTodo}
 />);
 const inputElement = screen.getByPlaceholderText(text: /add a new task here.../i)
 const buttonElement = screen.getByRole(text: "button", options: {name:/add/i})
 fireEvent.change(inputElement, options: {target: {value:"Go Grocery Shopping"}})
 fireEvent.click(buttonElement)
 expect(inputElement.value).toBe(expected: "");
}
);
```

# Tests d'intégration

- Ces tests permettent de vérifier le comportement de plusieurs composantes oeuvrant ensemble. Par exemple, on veut voir les tasks s'inscrire dans le composant du dessous après un ajout via le composant du dessus



The image shows a screenshot of a web application with a green header containing the word 'TODO' in white. Below the header, there are two white rectangular components. The top component contains an input field with the text 'task3' and a green button labeled 'Add'. The bottom component contains a list of tasks: 'task1' and 'task2', each followed by a horizontal line for editing or deletion.

# Tests d'intégration

```
const addTasks = (tasks) => {
 const inputElement = screen.getByPlaceholderText(text: /add a new task here...
 const buttonElement = screen.getByRole(text: "button", options: { name: /Add/i}
 tasks.forEach(task => {
 fireEvent.change(inputElement, options: {target: {value: task}})
 fireEvent.click(buttonElement);
 })
}

describe(name: 'Todo', fn: () => {

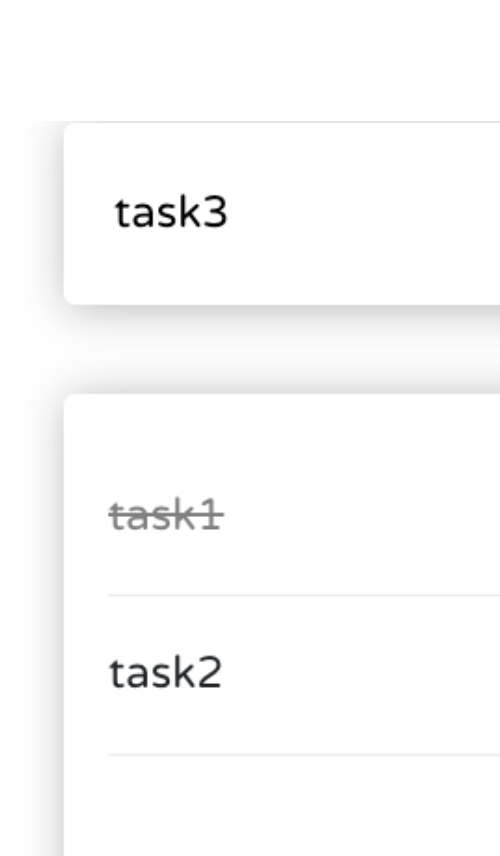
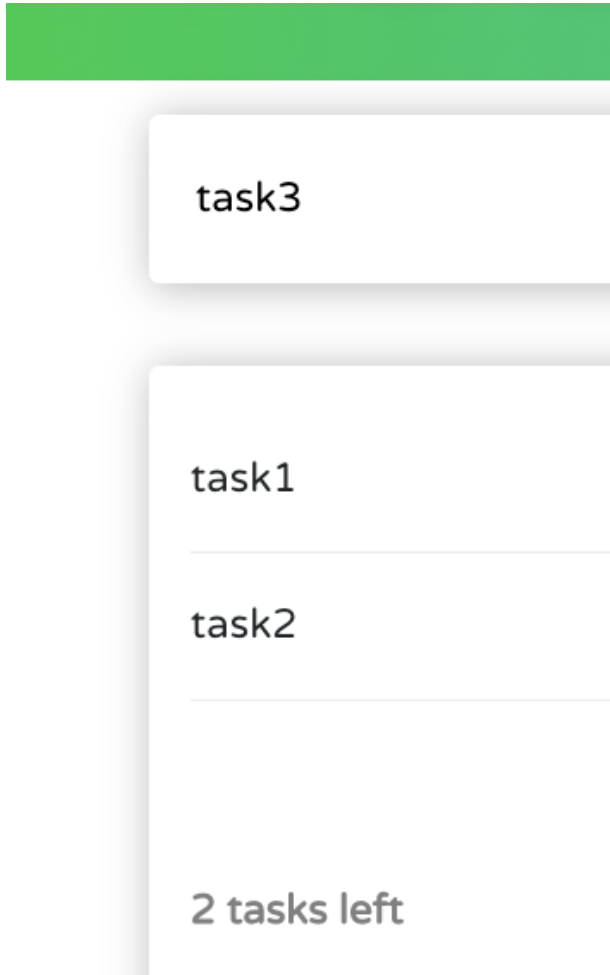
 it(name: 'test input adds to list', fn: async () => {
 render(<MockTodo/>);
 addTasks(tasks: ["Go Grocery Shopping"])
 const divElement = screen.getByText(text: /Go Grocery Shopping/i)
 expect(divElement).toBeInTheDocument();
 });
});
```

# Test d'intégration

```
it(name: 'test input adds more to list', fn: async () => {
 render(<MockTodo/>);
 addTasks(tasks: ["Go Grocery Shopping", "Wash my hands"])
 const divElement = screen.getAllByTestId(text: "task-container")
 expect(divElement.length).toBe(expected: 2);
});

<div>
 {
 todos.map((todo, index) => (
 <div
 className={`todo-item ${todo.completed} && "todo-item-active"}`}
 onClick={() => updateTask(todo.id)}
 key={index}
 data-testid="task-container" You, 2 minutes ago • Uncommitted
 >
 {todo.task}
 </div>
))
 }
</div>
```

# Tests de formatage (css)



# Test de formatage (css)

```
{
 todos.map((todo, index) => (
 <div
 className={`todo-item ${todo.completed} && "todo-item-active"}`}
 onClick={() => updateTask(todo.id)}
 key={index}
 data-testid="task-container"
 >
 {todo.task}
 </div>
))
}

it(name: 'task should not have completed when initially rendered', fn: async () => {
 render(<MockTodo/>);
 addTasks(tasks: ["Go Grocery Shopping"])
 const divElement = screen.getByText(text: /go grocery shopping/i)
 expect(divElement).not.toHaveClass(classNames: "todo-item-active");
});

it(name: 'task should completed class when clicked', fn: async () => {
 render(<MockTodo/>);
 addTasks(tasks: ["Go Grocery Shopping"])
 const divElement = screen.getByText(text: /go grocery shopping/i)
 fireEvent.click(divElement)
 expect(divElement).toHaveClass("todo-item-active");
});
```

# Tests asynchrones

- Allez sur le lien suivant <http://localhost:3001/followers>

## FOLLOWERS



Abella Pires

ticklishcat371



Melike Kaya

beautifulsnake137



Monali Shayana

vellowleopard104

# Tests asynchrones

```
export default function FollowersList() {

 const [followers, setFollowers] = useState(initialState: []);

 useEffect(effect: () => {
 fetchFollowers()
 }, deps: []);

 const fetchFollowers = async () => {
 const {data} = await axios.get(url: "https://randomuser.me/api/?results=5")
 setFollowers(data.results)
 }

 return (
 <div className="followerslist-container">
 <div>
 {followers.map(follower => (
 <div className="follower-item">

 <div className="followers-details">
 <div className="follower-item-name">
 <h4>{follower.name.first}</h4> <h4>{follower.name.last}</h4>
 </div>
 <p>{follower.login.username}</p>
 </div>
 </div>
))}
 </div>
 </div>
)
}
```

# Mock

- Comment avoir des mocks en js
- Exemple de mock pour Axios

```
export default function FollowersList() {

 const [followers, setFollowers] = useState(initialState: []);

 useEffect(effect: () => {
 fetchFollowers()
 }, deps: []);

 const fetchFollowers = async () => {
 const {data} = await axios.get(url: "https://randomuser.me/api/?results=5")
 setFollowers(data.results)
 }
}
```

Harb, 2021-06-19, 19:03 • Complete starter project

# Mock

```
import axios from "axios";

export const fetchFollowers = async (url) => {
 try {
 return await axios.get(url);
 } catch (e) {
 return [];
 }
};
```

```
const fetchFollowers = async () => {
 //const {data} = await axios.get("https://randomuser.me/api/?results=5")
 const {data} = await FetchService.fetchFollowers(url: "https://randomuser.me/api/?results=5")
 setFollowers(data.results)
}
```

# Mock

```
import axios from "axios";
import {fetchFollowers} from '../utils/FetchService'

jest.mock(moduleName: "axios");

const mockurl = "https://mockurl.com/someusers"
const mockResponse = {
 data: {
 results: [
 {
 name: {
 first: "firstname",
 last: "lastname"
 },
 picture: {
 large: "https://someaddress.com/pic.jpg"
 },
 login: {
 username: "username"
 }
 }
]
 }
}
```

# Mock

```
describe(name: 'fetchFollowers happy day', fn: () => {
 it(name: 'should return a list of followers', fn: async () => {
 // Arrange
 axios.get.mockResolvedValueOnce(mockResponse)
 // Act
 const result = await fetchFollowers(url: `${mockurl}`)
 // Assert
 expect(axios.get).toHaveBeenLastCalledWith(params: `${mockurl}`);
 expect(result).toEqual(mockResponse)
 })
})

describe(name: 'fetchFollowers fails', fn: () => {
 it(name: 'should return an empty list', fn: async () => {
 // Arrange
 axios.get.mockRejectedValueOnce(new Error("network error"))
 // Act
 const result = await fetchFollowers(url: `${mockurl}`)
 // Assert
 expect(axios.get).toHaveBeenCalledWith(params: `${mockurl}`);
 expect(result).toEqual(expected: []);
 })
})
```